

Spring boot - 7

7. REST 서비스 사용하기

RestTemplate : Spring Framework에서 제공하는 간단하고 동기화된 클라이언트

Traverson : Spring HATEOAS에서 제공하는 하이퍼링크를 인식하는 동기화 REST 클라이언트로 같은 이름의 자바스크립트 라이브러리로부터 비롯

WebClient : Spring 5에서 소개된 반응형 비동기 REST 클라이언트

7.1 RestTemplate으로 REST 엔드포인트 사용하기

클라이언트 입장에서 REST 리소스와 상호작용하려면 해야 할 일이 많아서 코드가 장황해진다.

즉, 저수준의 HTTP 라이브러리로 작업하면서 클라이언트는 클라이언트 인스턴스와 요청 객체 생성하고, 해당 요청을 실행하고, 응답을 분석하여 관련 도메인 객체와 연관시켜서 처리해야한다. 그 와중에 발생할 수 있는 예외도 처리해야한다.

이러한 장황한 코드를 피하기 위해 스프링은 **RestTemplate**을 제공한다.

JDBC를 사용할 때 번거로운 작업을 JdbcTemplate이 처리하듯이, RestTemplate은 REST 리소스를 사용하는데 번잡한 일을 처리해준다.

RestTemplate은 REST 리소스와 상호작용하기 위한 **41개의 메소드**를 제공한다.

그렇지만 고유한 작업을 수행하는 메소드는 **12개**이며, 나머지는 이 메소드들의 오버로딩된 버전이다.

RestTemplate이란?

- Spring이 제공하는 **동기식 HTTP 클라이언트**
- 자바 코드에서 **다른 REST API 서버에 요청/응답 도와줌**
- 내부적으로 HttpURLConnection 또는 Apache HttpClient 같은 라이브러리 래핑해서 더 쉽게 사용 가능하게 만든 것
- TRACE를 제외한 표준 HTTP 메소드 각각에 대해 최소한 하나의 메소드를 갖고 있다

execute()와 exchange()는 모든 HTTP 요청을 전송하기 위한 저수준의 범용 메소드를 제공한다.

메소드 오버로딩 형태

앞으로 소개할 메소드는 다음의 세 가지 형태로 오버로딩 되어있다.

1. 가변 인자 리스트에 지정된 URL 매개변수에 URL 문자열(String 타입)을 인자로 받는다
2. Map<String, String>에 지정된 URL 매개변수에 URL 문자열을 인자로 받는다
3. java.net.URL에 대한 인자로 받으며, 매개변수화된 URL은 지원하지 않는다

RestTemplate 사용법

RestTemplate을 사용하려면 우리가 필요한 시점에 RestTemplate 인스턴스를 생성해야한다.

```
RestTemplate rest = new RestTemplate();
```

또는 빈으로 선언하고 필요할 때 주입할 수 있다.

```
@Bean
public RestTemplate restTemplate(){
    return new RestTemplate();
}
```

7.1.1 GET 요청

HATEOAS가 활성화되지 않았다면 `getForObject()` 사용할 수 있다.

getForObject()

GET 요청 후, 응답을 객체로 바로 반환 (응답 Body 필요 시)

```
getForObject(String url, Class<T> responseType, Object...uriVariables)
: 지정한 URL에 GET 요청 → 응답바디만 반환
: Object...uriVariables : URL 파라미터(순서 존재)
```

가변 인자 리스트 사용 예제

```
public Ingredient getIngredientById(String ingredientId) {
    return rest.getForObject("http://localhost:8080/ingredients/{id}",
                            Ingredient.class,
                            ingredientId);
}
```

- 여기서는 URL 변수의 가변 리스트와 URL 문자열을 인자로 받게해서 오버로딩된 `getForObject()` 사용했다.
- `ingredientId`는 `{id}`에 들어가며 변수 매개변수들은 주어진 순서대로 지정한다.
- Class `responseType`는 응답이 바인딩 되는 타입으로 JSON 형식인 응답 데이터가 객체로 역직렬화되어 반환한다

Map 사용 예제

Map 사용해서 URL 지정이 가능하다

```
public Ingredient getIngredientById(String ingredientId) {
    Map<String, String> params = new HashMap<>();
    params.put("id", ingredientId);

    return rest.getForObject("http://localhost:8080/ingredients/{id}",
                            Ingredient.class,
                            params);
}
```

```
// Key는 id이며 요청이 수행될 때 {id}에 Key가 id인 Map 항목 값으로 교체된다
}
```

URI 객체 사용 예제

이와 달리 URI 매개변수를 사용할 때는 URI 객체를 구성하여 `getForObject()`를 호출하여야한다.

```
public Ingredient getIngredientById(String ingredientId){
    Map<String, String> uriVariables = new HashMap<>();
    uriVariables.put("id", ingredientId);

    URI url = UriComponentsBuilder
        .fromHttpUrl("http://localhost:8080/ingredients/{id}")
        .build(uriVariables);

    return rest.getForObject(url, Ingredient.class);
}
```

UriComponentsBuilder 주요 메소드

fromHttpUrl(String uriTemplate)

- URI 템플릿을 문자열로부터 시작해 URI 조립 시작한다
- 역할: URI 빌더의 기초 주소 지정
- 입력: <http://api.com/users/{id}> 같이 {} 템플릿 가능
- 반드시 http:// 혹은 https:// 형식 포함해야한다

buildAndExpand()

- {id} 같은 URI 경로 템플릿에 실제 값 치환한다
- 역할: {id} → value로 변환하여 전체 URI를 만든다
- 입력: Map<String, String> 형식으로 { 변수명 = 값 } 구조 넘긴다
- build().expand(value) 혹은 build().expand("id", value) 가능하다

encode()

- URI 전체를 RFC 3986에 따라 인코딩
- encode()는 buildAndExpand() 이후에 사용하는 것이 권장된다

toUri()

- 최종적으로 java.net.URI 객체로 변환한다
- 사용처: RestTemplate, WebClient, HttpHeaders 등 사용가능

getForEntity()

그러나 클라이언트가 이외에 추가로 필요한 것이 있다면 `getForEntity()` 사용할 수 있다.

GET 요청 후, 응답(ResponseEntity) 반환 (헤더 + 상태코드 포함 필요시)

```
getForEntity(String url, Class<T> responseType, Object...uriVariables)
```

- : 지정한 URL에 GET 요청
- : `ResponseEntity`(상태 코드 + 헤더 + 바디) 반환
- : 상태코드 / 헤더까지 필요할 때 사용한다

7.1.2 POST 요청

POST 요청이 수행된 후, 새로 생성된 객체의 리소스를 반환받고 싶다면 `postForObject()`를 사용한다

`postForObject()`

POST 요청 후, 응답을 객체로 반환 (form 전송 혹은 JSON 전송에 사용)

```
postForObject(String url, Object request, Class<T> responseType,
Object...uriVariables)
```

- : 지정한 URL에 POST 요청 (바디 포함) → 응답 바디 반환

예제

```
public Ingredient createIngredient(Ingredient ingredient){
    return rest.postForObject("http://localhost:8080/ingredients",
                              ingredient,
                              Ingredient.class);
}
```

`postForObject()`는 메소드 문자열(String 타입) URL과 서버에 전송될 객체 및 이 객체의 타입(리소스 몸체의 데이터와 연관된) 인자를 받는다

`postForEntity()`

POST 요청 후, 전체 응답 반환 (상태코드 확인이 필요한 경우)

```
postForEntity(String url, Object request, Class<T> responseType,
Object...uriVariables)
```

- : 지정한 URL에 POST 요청 → `ResponseEntity` 반환

예제

```
User user = restTemplate.postForObject(URL, new User("JM"), User.class);

ResponseEntity<User> response = restTemplate.postForEntity(
    URL,
    new User("JM"),
    User.class
);
```

7.1.3 POST 후, Location URL 반환

postForLocation()

postForLocation(String url, Object request, Object...uriVariables)

- 지정한 URL에 POST 요청한다
- 새로 생성된 리소스의 URI 반환
- 보통 201 응답에서 Location 헤더 추출할 때 사용한다

예제

```
URI location = restTemplate.postForLocation(  
    URL,  
    new User("jm")  
);
```

postForObject()와 postForLocation()은 동일하게 작동하지만 리소스 객체 대신 새로 생성된 리소스의 URI를 반환한다는 것이 다르다.

반환된 URI는 해당 응답의 Location 헤더에서 얻는다.

만일 새로 생성된 리소스의 위치와 리소스 객체 모두 필요하다면 postForEntity()를 호출하면 된다.

7.1.4 PUT 요청

HTTP PUT 요청을 전송하기 위해 RestTemplate은 put() 메소드를 제공한다

put()

put(String url, Object request, Object...uriVariables)

- 지정한 URL에 PUT 요청 : 응답이 없음(void)
- 리소스 전체 교환 시 사용한다

7.1.5 DELETE 요청

delete()

delete(String url, Object...uriVariables)

- 지정한 URL에 DELETE 요청하며 마찬가지로 응답이 없다(void)
-

7.1.6 기타 메소드

URI 조회

headForHeaders(String url, Object...uriVariables)

- HEAD 요청 보내고 헤더 정보만 반환하며 보통 유효성 검사에 사용한다

OPTIONS 요청

optionsForAllow(String url, Object...uriVariables)

- 지정한 URL에 OPTIONS 요청 보내고 허용된 HTTP 메소드 목록 반환한다

예제

```
Set<HttpMethod> methods = restTemplate.optionsForAllow("http://example.com/api");
```

URI 추출

getForLocation()

- GET 요청 후 Location 헤더 반환한다

일반 요청

exchange()

- 모든 HTTP 요청을 처리할 수 있는 범용 메소드
- GET, POST, PUT 등 지정 가능하며 헤더/바디/상태코드/응답 모두 제어 가능하다

예제

```
HttpHeaders headers = new HttpHeaders();
headers.set("Authorization", "Bearer token");
HttpEntity<String> entity = new HttpEntity<>(headers);

ResponseEntity<String> response = restTemplate.exchange(
    "http://example.com/api", // URL
    HttpMethod.GET,
    entity,
    String.class
);
```

요청 - 응답 템플릿

execute()

- 가장 저수준 요청 실행 메소드로, 콜백을 통해 요청/응답 커스터마이징한다

7.2 외부 API 호출 작성법

외부 API 연동하여 작은 프로젝트 만들면서 공부한 내용을 정리했다.

7.2.1 Header 설정

외부 API는 보통 인증, 요청 타입 지정 등 위해 헤더를 설정해야한다
HttpHeaders 객체 사용하며 Key-Value 형태로 추가한다

```
HttpHeaders headers = new HttpHeaders();
headers.set("Authorization", "Bearer " + token);
headers.set("x-ncp-apigw-api-key-id", clientId);
// ...
headers.setContentType(MediaType.APPLICATION_JSON); // 요청 JSON 전송
headers.setAccept(List.of(MediaType.APPLICATION_JSON)); // 응답 JSON 수락
```

7.2.2 URL/쿼리 파라미터 설정

URL과 쿼리 파라미터를 설정하는 방법은 두 가지가 있다

- 직접 URLEncoder.encode() 쓰는 방법
- UriComponentsBuilder 쓰는 방법

UriComponentsBuilder 사용 예제

```
String url = UriComponentsBuilder.fromHttpUrl("http://api.example.com/route")
    .queryParams("start", start)
    .queryParams("goal", goal)
    .build()
    .encode()
    .toUriString();
```

URLEncoder.encode() 직접 안써도 인코딩 처리

7.2.3 요청 방식별 HttpMethod 전송

GET 요청

```
HttpEntity<?> entity = new HttpEntity<>(headers);
ResponseEntity<MyResponse> response = restTemplate.exchange(
    url,
    HttpMethod.GET,
    entity,
    MyResponse.class
);
```

매개변수 설명

- url : 호출할 URL
- HttpMethod.GET : 요청방식
- MyResponse.class : 응답 JSON을 어떤 타입으로 변환할지

entity 설명

- 요청 헤더 + 요청 바디 담은 객체
- new HttpEntity<?>(headers) : 헤더만
- new HttpEntity<?>(body, headers) : 바디 + 헤더

POST 요청

```
MyRequest req = new MyRequest("foo", "bar");
HttpEntity<MyRequest> entity = new HttpEntity<>(req, headers);

ResponseEntity<MyResponse> response = restTemplate.exchange(
    url,
    HttpMethod.POST,
    entity,
    MyResponse.class
);
```
